



UNIVERSITÀ DEGLI STUDI DI PERUGIA

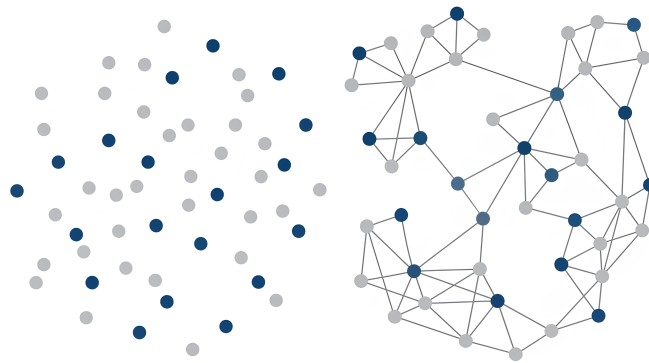
Dipartimento di Ingegneria

Corso di Laurea Magistrale in Ingegneria Informatica e Robotica

A.A. 2025-2026

Tesina di
Signal Processing and Optimization for Big Data

Implementazione di un algoritmo di Graph Learning tramite Deep Unfolding



Docente:

Prof. Banelli Paolo

Studente:

Lattaioli Daniele (383459)

Indice

1	Introduzione	2
2	Formulazione del problema	3
3	Risoluzione del problema di ottimizzazione	5
3.1	Formulazione primale-duale	5
3.2	Condizioni di punto di sella	6
3.3	Schema forward-backward-forward	6
3.4	Calcolo esplicito dei termini	7
4	Implementazione dell'algoritmo	8
5	Sperimentazione	10
5.1	Generazione dei dati	10
5.2	Configurazione dell'architettura	12
5.2.1	Metriche	12
5.2.2	Algoritmo PDS	13
5.2.3	Rete neurale	14
5.2.4	Protocollo di confronto	15
5.3	Risultati ricostruzione grafi ER	16
5.4	Risultati ricostruzione grafi BA	17
5.5	Confronto ricostruzioni grafi ER e BA	21
6	Conclusioni	24

Capitolo 1

Introduzione

I grafi rappresentano un potente strumento matematico per la modellazione di relazioni fra entità e vengono utilizzati in molteplici scenari applicativi. Tuttavia, in molti contesti pratici non si ha effettivamente accesso alla struttura del grafo, ma soltanto ad un set ristretto di segnali definiti sopra lo stesso.

Il **Graph Learning** è la disciplina che, a partire dall'osservazione di segnali registrati sui nodi di un sistema, cerca di ricostruire la struttura invisibile delle relazioni che ha generato o regola quei segnali.

Nella letteratura moderna sono stati proposti molti algoritmi model-based per il Graph Learning, basati sulla risoluzione di un problema di ottimizzazione sopra lo spazio dei grafi. Tipicamente, la funzione obiettivo si compone di un termine legato alla fedeltà dei dati osservati e di un termine di regolarizzazione che impone proprietà strutturali del grafo. Tuttavia, i regolarizzatori "handcrafted" possono risultare non sufficientemente espressivi rispetto alla struttura di grafi complessi.

In questo studio si propone l'implementazione di un algoritmo per il Graph Learning tramite l'approccio **Deep Unfolding**. Sfruttando la struttura a strati di una rete neurale profonda (DNN), si può implementare la soluzione iterativa di un problema model-based. In questo modo, l'apprendimento della struttura del grafo è guidato dai dati raccolti sopra di esso, con il vantaggio di rendere il modello altamente interpretabile e più performante in applicazioni pratiche.

L'obiettivo di questo lavoro è quello di testare alcune delle soluzioni al problema del Graph Learning presentate all'interno del paper "Learning to Learn Graph Topologies" [3].

Capitolo 2

Formulazione del problema

Definiamo $\mathcal{G} = \{\mathcal{V}, \mathcal{E}, W\}$ un grafo non orientato pesato, dove $\mathcal{V}$ è l'insieme dei nodi con $|\mathcal{V}| = N$, $\mathcal{E}$ è l'insieme degli archi e W è la matrice di adiacenza contenente la similarità fra i nodi.

La matrice laplaciana è definita come $L = D - W$, dove D è la matrice diagonale contenente i gradi dei singoli nodi.

Si suppone di avere accesso ad una matrice di dati $X = [\underline{x}_1, \underline{x}_2, \dots, \underline{x}_Q] \in R^{N \times Q}$, dove ogni colonna $\underline{x}_i$ rappresenta un segnale osservato sopra al grafo.

La struttura del grafo è completamente caratterizzata dalla matrice Laplaciana L o dalla matrice di adiacenza pesata W , per cui un tipico problema di Graph Learning viene modellato come:

$$\begin{aligned} \min_L \quad & \{\text{trace}(X^T L X) + \Omega(L)\} \\ \text{s.t.} \quad & L \underline{1} = \underline{0}, \quad L_{ij} = L_{ji} \leq 0, \quad i \neq j \\ & \text{trace}(L) = N \end{aligned}$$

- **trace($X^T L X$)** : rappresenta la Total Variation di X sopra il grafo

$$\text{TV}(X) = \sum_{q=1}^Q \text{TV}(\underline{x}_q) = \sum_{q=1}^Q \frac{1}{2} \sum_{i,j} w_{ij} (x_q^{(i)} - x_q^{(j)})^2 = \text{trace}(X^T L X)$$

- **$\Omega(L)$** : rappresenta un regolarizzatore convesso su L che riflette le proprietà strutturali. Ad esempio, nel Graphical LASSO $\Omega(L)$ corrisponde alla norma l_1 della matrice di precisione P_x :

$$\hat{P}_x = \arg \max_{P_x \succ 0} \left\{ \ln(|P_x|) - \text{trace}(\hat{\Sigma}_x P_x) - \underbrace{\lambda \|P_x\|_1}_{\Omega} \right\}$$

Il problema può essere riformulato introducendo il vettore $\underline{w} \in R^{N(N-1)/2}$ dei pesi degli archi:

$$\text{trace}(X^T L X) = \sum_{i,j} w_{ij} \|\underline{x}_i - \underline{x}_j\|^2 = 2\underline{w}^T \underline{y}$$

dove $\underline{y}$ è la half-vectorisation della matrice delle distanze euclidee:

$$Z_{ij} = \|\underline{x}^{(i)} - \underline{x}^{(j)}\|_2^2 = \sum_{q=1}^Q (x_q^{(i)} - x_q^{(j)})^2$$

Il problema diventa quindi:

$$\min_{\underline{w}} 2\underline{w}^T \underline{y} + \mathcal{I}_{\underline{w} \geq 0}(\underline{w}) + \Omega_1(\underline{w}) + \Omega_2(\mathcal{D}\underline{w})$$

dove:

- $\mathcal{I}_{\underline{w} \geq 0}(\underline{w})$: rappresenta la funzione indicatrice

$$\mathcal{I}_{\{\underline{w} \geq 0\}}(\underline{w}) = \begin{cases} 0 & \text{se } w_{ij} \geq 0 \quad \forall i, j \\ +\infty & \text{altrimenti} \end{cases}$$

- $\Omega_1(\underline{w})$: rappresenta il regolarizzatore per i pesi degli archi
- $\Omega_2(\mathcal{D}\underline{w})$: rappresenta il regolarizzatore per i gradi dei nodi, dove $\mathcal{D}$ è un operatore lineare che trasforma $\underline{w}$ nel vettore dei gradi dei nodi

$$\mathcal{D}\underline{w} = W\underline{1}$$

Tipicamente [1] i due regolarizzatori vengono impostati come segue:

- $\Omega_1(\underline{w}) = \|\underline{w}\|_2^2$: controlla la sparsità/densità del grafo, penalizzando pesi grandi e distribuendo la massa su più archi
- $\Omega_2(\mathcal{D}\underline{w}) = -\underline{1}^T \log(\mathcal{D}\underline{w})$: log-barrier, previene che il grafo appreso presenti nodi isolati

Il problema di ottimizzazione risultante diventa:

$$\boxed{\min_{\underline{w}} 2\underline{w}^T \underline{y} + \mathcal{I}_{\{\underline{w} \geq 0\}}(\underline{w}) - \alpha \underline{1}^T \log(\mathcal{D}\underline{w}) + \beta \|\underline{w}\|_2^2} \quad (2.1)$$

Capitolo 3

Risoluzione del problema di ottimizzazione

Il problema di ottimizzazione (2.1) può essere risolto utilizzando un algoritmo iterativo **Primal-Dual Splitting (PDS)** di tipo *forward-backward-forward* [2].

In particolare, l'algoritmo scinde i termini differenziabili da quelli non differenziabili; nel primo caso, si effettua una discesa del gradiente, nel secondo, si utilizzano invece operatori prossimali.

$$\min_{\underline{w}} \underbrace{2\underline{w}^T \underline{y} + \beta \|\underline{w}\|_2^2}_{f(\underline{w}) \text{ Differenziabile}} + \underbrace{\mathcal{I}_{\{\underline{w} \geq 0\}}(\underline{w})}_{g(\underline{w})} - \underbrace{\alpha \underline{1}^T \log(\mathcal{D}\underline{w})}_{h(\mathcal{D}\underline{w})} \quad (3.1)$$

La forma canonica del primal-dual splitting è la seguente:

$$\min_{\underline{w}} f(\underline{w}) + g(\underline{w}) + h(\mathcal{D}\underline{w})$$

dove $f(\underline{w})$ è differenziabile, mentre $g(\underline{w})$ e $h(\mathcal{D}\underline{w})$ non sono differenziabili.

Il termine problematico è $h(\mathcal{D}\underline{w})$. Sebbene l'operatore prossimale di h sia noto in forma chiusa, quello della composizione $h \circ \mathcal{D}$ non lo è. L'approccio primale-duale serve precisamente a disaccoppiare h da $\mathcal{D}$.

3.1 Formulazione primale-duale

Poiché h è convessa, chiusa e propria, vale la biconiugazione $h^{**} = h$, ovvero

$$h(\underline{z}) = \sup_{\underline{v}} \{ \langle \underline{z}, \underline{v} \rangle - h^*(\underline{v}) \}$$

dove h^* è la coniugata di Fenchel di h . Sostituendo $\underline{z} = \mathcal{D}\underline{w}$ si ottiene la formulazione primale-duale:

$$\min_{\underline{w}} \max_{\underline{v}} \underbrace{f(\underline{w}) + g(\underline{w}) + \langle \mathcal{D}\underline{w}, \underline{v} \rangle - h^*(\underline{v})}_{\Lambda(\underline{w}, \underline{v})}$$

Ora $\mathcal{D}\underline{w}$ compare soltanto all'interno di un prodotto scalare, quindi in modo lineare, e può essere gestito con un semplice passo di gradiente anziché con un operatore prossimale. Il prezzo da pagare è l'introduzione della variabile duale $\underline{v} \in R^N$, che nel problema in esame è associata al vettore dei gradi dei nodi, essendo $\mathcal{D}\underline{w} = W\underline{1}$.

La funzione $\Lambda(\underline{w}, \underline{v})$ è convessa in $\underline{w}$ e concava in $\underline{v}$, per cui la sua soluzione $(\underline{w}^*, \underline{v}^*)$ è un punto di sella: un minimo lungo la direzione primale e un massimo lungo quella duale.

3.2 Condizioni di punto di sella

Un punto di sella è caratterizzato dall'annullarsi del subdifferenziale rispetto a ciascuna variabile separatamente.

Si ottengono così le due condizioni:

$$0 \in \nabla_{\underline{w}} f(\underline{w}) + \partial g(\underline{w}) + \mathcal{D}^T \underline{v} \quad \text{e} \quad 0 \in \mathcal{D}\underline{w} - \partial h^*(\underline{v})$$

Le due condizioni sono accoppiate: la prima coinvolge $\underline{v}$ e la seconda $\underline{w}$. Non è quindi possibile risolverle in sequenza, e questo motiva il ricorso ad uno schema iterativo che aggiorni congiuntamente le due variabili.

3.3 Schema forward-backward-forward

Ponendo $\underline{x} = (\underline{w}, \underline{v})$, le condizioni di punto di sella si riscrivono come un'unica condizione $0 \in A(\underline{x}) + B(\underline{x})$, dove

$$A(\underline{x}) = (\partial g(\underline{w}), \partial h^*(\underline{v})) \quad B(\underline{x}) = (\nabla f(\underline{w}) + \mathcal{D}^T \underline{v}, -\mathcal{D}\underline{w})$$

A raccoglie i termini non differenziabili, trattati per via prossimale (passo *backward*), mentre B raccoglie quelli differenziabili, trattati esplicitamente (passo *forward*).

Lo schema forward-backward-forward proposto da Tseng applica un passo esplicito, uno prossimale ed un secondo passo esplicito, seguiti da una correzione:

$$\begin{aligned} \underline{r}^{(k)} &= \underline{x}^{(k)} - \gamma B(\underline{x}^{(k)}) \\ \underline{p}^{(k)} &= \text{prox}_{\gamma A}(\underline{r}^{(k)}) \\ \underline{q}^{(k)} &= \underline{p}^{(k)} - \gamma B(\underline{p}^{(k)}) \\ \underline{x}^{(k+1)} &= \underline{x}^{(k)} - \underline{r}^{(k)} + \underline{q}^{(k)} \end{aligned}$$

Esplicitando le componenti si ottiene l'algoritmo complessivo:

$$\begin{aligned}
\underline{r}_1^{(k)} &= \underline{w}^{(k)} - \gamma \left(\nabla f(\underline{w}^{(k)}) + \mathcal{D}^T \underline{v}^{(k)} \right) \\
\underline{r}_2^{(k)} &= \underline{v}^{(k)} + \gamma \mathcal{D} \underline{w}^{(k)} \\
\underline{p}_1^{(k)} &= \text{prox}_{\gamma g} \left(\underline{r}_1^{(k)} \right) \\
\underline{p}_2^{(k)} &= \text{prox}_{\gamma h^*} \left(\underline{r}_2^{(k)} \right) \\
\underline{q}_1^{(k)} &= \underline{p}_1^{(k)} - \gamma \left(\nabla f(\underline{p}_1^{(k)}) + \mathcal{D}^T \underline{p}_2^{(k)} \right) \\
\underline{q}_2^{(k)} &= \underline{p}_2^{(k)} + \gamma \mathcal{D} \underline{p}_1^{(k)} \\
\underline{w}^{(k+1)} &= \underline{w}^{(k)} - \underline{r}_1^{(k)} + \underline{q}_1^{(k)} \\
\underline{v}^{(k+1)} &= \underline{v}^{(k)} - \underline{r}_2^{(k)} + \underline{q}_2^{(k)}
\end{aligned}$$

3.4 Calcolo esplicito dei termini

Essendo $f(\underline{w}) = 2\underline{w}^T \underline{y} + \beta \|\underline{w}\|_2^2$, il gradiente vale:

$$\nabla_{\underline{w}} f(\underline{w}) = 2\underline{y} + 2\beta \underline{w}$$

Il primo operatore prossimale è quello dell'indicatrice $g = \mathcal{I}_{\{\underline{w} \geq 0\}}$, che coincide con la proiezione sull'ortante non negativo:

$$\underline{p}_1^{(k)} = \text{prox}_{\gamma g} \left(\underline{r}_1^{(k)} \right) = \max\{0, \underline{r}_1^{(k)}\}$$

Il secondo riguarda invece la coniugata di $h(\underline{z}) = -\alpha \underline{1}^T \log(\underline{z})$ e si dimostra essere pari a:

$$(\underline{p}_2^{(k)})_i = \left(\text{prox}_{\gamma h^*} \left(\underline{r}_2^{(k)} \right) \right)_i = \frac{r_{2,i} - \sqrt{r_{2,i}^2 + 4\alpha\gamma}}{2}$$

Capitolo 4

Implementazione dell'algoritmo

Per valutare l'efficacia dell'approccio proposto sono state realizzate due implementazioni dello stesso schema di ottimizzazione. Nella prima, l'algoritmo PDS forward-backward-forward viene eseguito nella sua forma iterativa classica, con un criterio di arresto basato sulla variazione relativa della soluzione e con gli iperparametri α, β, γ selezionati tramite grid search. Nella seconda, un numero fissato K di iterazioni viene srotolato, interpretando ciascuna iterazione come uno strato di una rete neurale i cui parametri vengono appresi end-to-end minimizzando una funzione di perdita supervisionata.

Algoritmo 1 PDS

```
1: Input:  $\underline{\mathbf{y}}, \gamma, \alpha$  and  $\beta$ .
2: Initialisation:  $\underline{\mathbf{w}}^{(0)} = \underline{\mathbf{0}}, \underline{\mathbf{v}}^{(0)} = \underline{\mathbf{0}}$ 
3: while  $\|\underline{\mathbf{w}}^{(t)} - \underline{\mathbf{w}}^{(t-1)}\| > \epsilon$  do
4:    $\underline{\mathbf{r}}_1^{(t)} = \underline{\mathbf{w}}^{(t)} - \gamma(2\beta\underline{\mathbf{w}}^{(t)} + 2\underline{\mathbf{y}} + \mathcal{D}^\top \underline{\mathbf{v}}^{(t)})$ 
5:    $\underline{\mathbf{r}}_2^{(t)} = \underline{\mathbf{v}}^{(t)} + \gamma\mathcal{D}\underline{\mathbf{w}}^{(t)}$ 
6:    $\underline{\mathbf{p}}_1^{(t)} = \text{prox}_{\Omega_1}(\underline{\mathbf{r}}_1^{(t)}) = \max\{\underline{\mathbf{0}}, \underline{\mathbf{r}}_1^{(t)}\}$ 
7:    $\underline{\mathbf{p}}_2^{(t)} = \text{prox}_{\gamma, \Omega_2}(\underline{\mathbf{r}}_2^{(t)})$ , where
8:      $\left(\text{prox}_{\gamma, \Omega_2}(\underline{\mathbf{r}}_2^{(t)})\right)_i = \frac{r_{2,i}^{(t)} - \sqrt{(r_{2,i}^{(t)})^2 + 4\alpha\gamma}}{2}$ 
9:    $\underline{\mathbf{q}}_1^{(t)} = \underline{\mathbf{p}}_1^{(t)} - \gamma(2\beta\underline{\mathbf{p}}_1^{(t)} + 2\underline{\mathbf{y}} + \mathcal{D}^\top \underline{\mathbf{p}}_2^{(t)})$ 
10:   $\underline{\mathbf{q}}_2^{(t)} = \underline{\mathbf{p}}_2^{(t)} + \gamma\mathcal{D}\underline{\mathbf{p}}_1^{(t)}$ 
11:   $\underline{\mathbf{w}}^{(t+1)} = \underline{\mathbf{w}}^{(t)} - \underline{\mathbf{r}}_1^{(t)} + \underline{\mathbf{q}}_1^{(t)}$ 
12:   $\underline{\mathbf{v}}^{(t+1)} = \underline{\mathbf{v}}^{(t)} - \underline{\mathbf{r}}_2^{(t)} + \underline{\mathbf{q}}_2^{(t)}$ 
13:   $t \leftarrow t + 1$ 
14: end while
15: return  $\widehat{\underline{\mathbf{w}}} = \underline{\mathbf{w}}^{(t)}$ 
```

Algoritmo 2 Unrolling Net

```
1: Input:  $\underline{\mathbf{y}}, T, \text{shared} \in \{\text{true}, \text{false}\}$ 
2: Initialisation:  $\underline{\mathbf{w}}^{(0)} = \underline{\mathbf{0}}, \underline{\mathbf{v}}^{(0)} = \underline{\mathbf{0}}$ 
3:  $\tau(t) \leftarrow 0$  se  $\text{shared} = \text{true}$ , altrimenti  $\tau(t) \leftarrow t$ 
4: for  $t = 0, 1, \dots, T - 1$  do
5:    $\underline{\mathbf{r}}_1^{(t)} = \underline{\mathbf{w}}^{(t)} - \gamma^{(\tau(t))} (2\beta^{(\tau(t))} \underline{\mathbf{w}}^{(t)} + 2\underline{\mathbf{y}} + \mathcal{D}^\top \underline{\mathbf{v}}^{(t)})$ 
6:    $\underline{\mathbf{r}}_2^{(t)} = \underline{\mathbf{v}}^{(t)} + \gamma^{(\tau(t))} \mathcal{D} \underline{\mathbf{w}}^{(t)}$ 
7:    $\underline{\mathbf{p}}_1^{(t)} = \text{prox}_{\Omega_1}(\underline{\mathbf{r}}_1^{(t)}) = \max\{\underline{\mathbf{0}}, \underline{\mathbf{r}}_1^{(t)}\}$ 
8:    $\underline{\mathbf{p}}_2^{(t)} = \text{prox}_{\gamma^{(\tau(t))}, \Omega_2}(\underline{\mathbf{r}}_2^{(t)})$ , dove
9:     
$$\left( \text{prox}_{\gamma^{(\tau(t))}, \Omega_2}(\underline{\mathbf{r}}_2^{(t)}) \right)_i = \frac{r_{2,i}^{(t)} - \sqrt{\left(r_{2,i}^{(t)}\right)^2 + 4\alpha^{(\tau(t))}\gamma^{(\tau(t))}}}{2}$$

10:    $\underline{\mathbf{q}}_1^{(t)} = \underline{\mathbf{p}}_1^{(t)} - \gamma^{(\tau(t))} (2\beta^{(\tau(t))} \underline{\mathbf{p}}_1^{(t)} + 2\underline{\mathbf{y}} + \mathcal{D}^\top \underline{\mathbf{p}}_2^{(t)})$ 
11:    $\underline{\mathbf{q}}_2^{(t)} = \underline{\mathbf{p}}_2^{(t)} + \gamma^{(\tau(t))} \mathcal{D} \underline{\mathbf{p}}_1^{(t)}$ 
12:    $\underline{\mathbf{w}}^{(t+1)} = \underline{\mathbf{w}}^{(t)} - \underline{\mathbf{r}}_1^{(t)} + \underline{\mathbf{q}}_1^{(t)}$ 
13:    $\underline{\mathbf{v}}^{(t+1)} = \underline{\mathbf{v}}^{(t)} - \underline{\mathbf{r}}_2^{(t)} + \underline{\mathbf{q}}_2^{(t)}$ 
14: end for
15: return  $\underline{\mathbf{w}}^{(1)}, \underline{\mathbf{w}}^{(2)}, \dots, \underline{\mathbf{w}}^{(T)} = \underline{\widehat{\mathbf{w}}}$ 
```

La differenza sostanziale fra le due implementazioni risiede nei parametri α, β, γ . Infatti, mentre nell'Algoritmo 1 i parametri α, β, γ sono passati in input, nell'Algoritmo 2 diventano parametri addestrabili della rete neurale profonda. In questa configurazione, tramite il training set, la rete è in grado di regolare autonomamente α, β, γ eliminando il tuning manuale via grid search, adattando opportunamente l'intensità della regolarizzazione, e accelerando moltissimo la convergenza.

L'Algoritmo 2 presenta in input il parametro *shared* che consente di configurare la rete neurale affinché i parametri α, β, γ vengano condivisi o meno fra i diversi layer. Questo approccio, già testato dagli autori del paper [3], consente di incrementare le capacità rappresentative e migliora le performance in modo sostanziale. Nel seguito della trattazione ci si riferisce alle 2 versioni dell'Algoritmo 2 con la seguente nomenclatura:

- **Unrolling:** ciascun layer ha i propri parametri $\alpha^{(t)}, \beta^{(t)}, \gamma^{(t)}$ (*shared=false*)
- **Recurrent Unrolling:** ciascun layer condivide gli stessi parametri α, β, γ (*shared=true*)

Capitolo 5

Sperimentazione

5.1 Generazione dei dati

Per l'addestramento e la validazione degli algoritmi si è deciso di utilizzare le seguenti famiglie di grafi:

- Erdős–Rényi
- Scale-free

I grafi **Erdős–Rényi (ER)** sono dei grafi casuali. Definito il numero di nodi N , per ogni possibile coppia di nodi, si inserisce un arco che li connette con probabilità pari a p . Questo tipo di grafi non presenta una struttura rigida né proprietà topologiche particolari, e costituisce quindi un termine di paragone rispetto al quale isolare, sui grafi scale-free, il contributo effettivo dei prior topologici. La generazione è stata effettuata attraverso il modello Erdős–Rényi (ER), producendo grafi con $N = 50$ nodi ed una edge density pari a 0.078 (all'interno del range proposto dagli autori [3]).

I grafi **Scale-free** modellano una rete la cui distribuzione dei gradi segue una legge di potenza. Ciò significa che la maggior parte dei nodi possiede pochissimi collegamenti, mentre pochissimi nodi (hub) concentrano un numero elevatissimo di connessioni. I grafi scale-free modellano molto bene Internet, le reti sociali e le reti biologiche. La generazione è stata effettuata attraverso il modello Barabási-Albert (BA), producendo grafi con $N = 50$ nodi ed una edge density pari a 0.078.

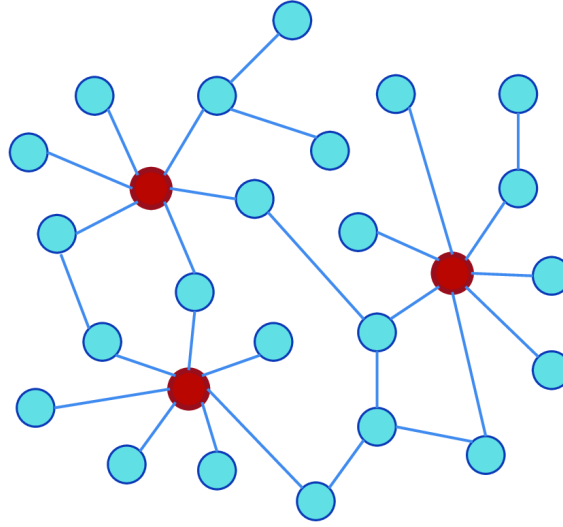


Figura 5.1: Esempio di grafo scale-free

I pesi degli archi sono stati campionati da una distribuzione log-normale $\log(W_{ij}) \sim \mathcal{N}(0, 0.1)$, dopodiché, la matrice di adiacenza pesata è stata impostata come:

$$W = \frac{(W + W^T)}{2}$$

per forzarne la simmetria.

I segnali definiti sopra i grafi sono stati invece generati campionando una Gaussiana:

$$X \sim \mathcal{N}(\underline{0}, K^{-1}) \quad \text{dove} \quad K = L + \sigma^2 I \quad \text{con} \quad \sigma = 0.01$$

Per ciascun grafo sono stati generati $n = 3000$ segnali. Il dataset complessivo è composto da 8000 coppie (y, w) per l'addestramento, 2000 per la validazione e 64 per il test, quest'ultimo dimensionato come in [3].

```

1  def sample(N=50, n_signals=3000, rng=None, graph_type='BA'):
2
3      # Generazione grafo connesso
4      while True:
5          if graph_type == 'BA':
6              G = nx.barabasi_albert_graph(N, 2,
↪ seed=int(rng.integers(1e9)))
7          elif graph_type == 'ER':
8              G = nx.erdos_renyi_graph(N, ER_DENSITY,
↪ seed=int(rng.integers(1e9)))
9          else:
10             raise ValueError(f"graph_type sconosciuto:
↪ {graph_type}")
11             if nx.is_connected(G):
12                 break
13

```

```

14     # Definizioni matrici A, W ed L
15     A = nx.to_numpy_array(G)
16     S = rng.normal(0, 0.1, (N,N))
17     W = np.exp(S) * A
18     W = (W + W.T) / 2
19     W = W * N / W.sum()
20     L = np.diag(W.sum(1)) - W
21
22     # Generazione segnali
23     P = L + 1e-4 * np.eye(N) # Precision matrix
24     cov = np.linalg.inv(P)
25     X = rng.multivariate_normal(np.zeros(N), cov, n_signals) #
    ↪ Segnali su grafo (n, N)
26
27     # Half-vectorisation
28     edM = euclidean_distances(X.T, squared=True) / n_signals # (N,
    ↪ N)
29     y = squareform(edM, checks=False) # (N(N-1)/2, 1)
30     w = squareform(W, checks=False)
31
32     return y, w

```

5.2 Configurazione dell'architettura

5.2.1 Metriche

I tre algoritmi implementati sono stati valutati nella ricostruzione dei grafi di test generati sinteticamente, utilizzando come metrica il **GMSE**:

$$\text{GMSE} = \frac{1}{n} \sum_{i=1}^n \frac{\|\hat{w}_i - w_i\|_2^2}{\|w_i\|_2^2}$$

Il GMSE misura l'accuratezza sui pesi degli archi, ma non è sensibile alla struttura topologica della stima: due ricostruzioni con GMSE simile possono corrispondere a grafi con proprietà molto diverse. Poiché la caratteristica che definisce i grafi scale-free è la **distribuzione dei gradi**, i risultati sono accompagnati dal confronto fra la distribuzione dei gradi dei grafi stimati e quella dei grafi di riferimento.

Il grado di un nodo i è $d_i = \sum_j 1[W_{ij} > 0]$, cioè il numero di suoi vicini. Per rendere il confronto indipendente dalla scelta di una soglia sui pesi (che penalizzerebbe i metodi che non producono zeri esatti) ogni stima viene sparsificata mantenendo i K archi di peso maggiore, con K pari al numero di archi del grafo di riferimento. In questo modo tutte le ricostruzioni hanno lo stesso numero di archi e differiscono solo per come questi sono distribuiti fra i nodi.

5.2.2 Algoritmo PDS

L'Algoritmo 1 è stato implementato all'interno della classe PDS e il cuore del suo funzionamento è racchiuso all'interno della funzione `solve()`. Una volta stabilito il numero di iterazioni sufficienti alla convergenza dell'algoritmo, come criterio di arresto è stato utilizzato il numero massimo di iterazioni `max_iter`.

```
1  def solve(self, y, max_iter=500):
2      y = torch.as_tensor(y, dtype=torch.float32)
3      E = y.shape[0]
4      N = int(round((1 + math.sqrt(1 + 8 * E)) / 2))
5      if not hasattr(self, '_D') or self._D.shape[0] != N:
6          self._D = make_D(N)
7      D = self._D
8
9      w = torch.zeros(E)
10     v = torch.zeros(N)
11
12     for _ in range(max_iter):
13         r1 = w - self.gamma * (2 * self.beta * w + 2 * y + D.T @ v)
14         r2 = v + self.gamma * (D @ w)
15
16         p1 = torch.clamp(r1, min=0.0)
17         p2 = prox2(r2, self.alpha, self.gamma)
18
19         q1 = p1 - self.gamma * (2 * self.beta * p1 + 2 * y + D.T @
↪ p2)
20         q2 = p2 + self.gamma * (D @ p1)
21
22         w = w - r1 + q1
23         v = v - r2 + q2
24
25     return w
```

La classe PDS è dotata inoltre della funzione `tune()` utilizzata per la ricerca dei parametri tramite grid search. Tale metodo è parametrico rispetto al numero di iterazioni, consente quindi di trovare i migliori parametri α, β, γ in funzione del numero di iterazioni previste per l'algoritmo.

Di seguito si riportano i parametri testati:

- $\alpha = [0.5, 1, 2, 4, 8, 16]$
- $\beta = [0.5, 1, 2, 4, 8, 16]$
- $\gamma = [0.03, 0.05, 0.1, 0.3, 0.5, 1.0, 2.0, 3.0]$

I valori ottimali risultanti dalla ricerca sono stati poi utilizzati per l'inizializzazione dei parametri della rete neurale a parità di numero di layer. Di seguito si riportano tali valori:

	ER			BA		
Iterazioni / layer	α	β	γ	α	β	γ
3	8	2	0.1	8	0.5	0.5
5	8	2	0.1	8	0.5	0.5
8	4	1	0.1	8	4	0.1
12	4	2	0.1	8	4	0.1
20	4	2	0.1	4	2	0.1
500 (convergenza)	2	2	0.3	2	2	0.1

Tabella 5.1: Risultati grid search per grafi ER e BA

La Tabella 5.1 mostra chiaramente come i parametri ottimali di PDS varino in funzione del numero di iterazioni previste.

5.2.3 Rete neurale

La rete neurale sviluppata per l'implementazione dell'Algoritmo 2 risulta configurabile rispetto al numero di layer (T) e alla modalità di gestione dei parametri.

```

1  class Unrolling(nn.Module):
2      def __init__(self, N, T=20, shared=False, alpha=2, beta=2,
    ↪ gamma=0.1):
3          super().__init__()
4          self.T = T
5          self.D = make_D(N)
6          n = 1 if shared else T
7          self.shared = shared
8
9          self._alpha = nn.Parameter(torch.full((n,),
    ↪ inv_softplus(float(alpha))))
10         self._beta = nn.Parameter(torch.full((n,),
    ↪ inv_softplus(float(beta))))
11         self._gamma = nn.Parameter(torch.full((n,),
    ↪ inv_softplus(float(gamma))))

```

Se `shared = True` la rete implementa di fatto la versione Recurrent Unrolling, altrimenti la variante Unrolling.

I parametri α, β, γ vengono inizialmente configurati ai valori trovati per PDS tramite grid search con un numero di iterazioni pari al numero di layer (vedi Tabella 5.1). Nel modello matematico questi tre valori hanno un significato preciso: α, β sono coefficienti di regolarizzazione e γ è il passo di discesa, tutti e tre sono positivi per costruzione. Per mantenere la spiegabilità dei parametri ed evitare che diventino negativi durante il training, invece di ottimizzare α direttamente, si ottimizza un parametro grezzo θ senza vincoli e si definisce $\alpha = \text{softplus}(\theta) = \log(1 + e^\theta)$, che è positivo per qualunque θ reale.

La funzione di loss utilizzata dalla rete neurale è la seguente:

$$\mathcal{L} = \sum_{t=1}^T \tau^{T-t} \frac{\|\underline{w}^{(t)} - \underline{\hat{w}}\|_2^2}{\|\underline{w}\|_2^2}$$

Gli aspetti interessanti della funzione di loss sono:

- La supervisione è su tutti i layer, non solo sull'ultimo. Ogni output intermedio $\underline{w}^{(t)}$ viene confrontato con la groundtruth.
- Il parametro $\tau \in (0, 1]$, che gli autori [3] definiscono *loss-discounting*, consente di ridurre il contributo della loss calcolata ai primi layer, quando le stime sono ancora grossolane. In questo caso è stato impostato $\tau = 0.9$.

Gli iperparametri di training utilizzati sono riportati nella tabella seguente.

Iperparametro	Valore
Ottimizzatore	Adam
Learning rate iniziale	10^{-2}
Scheduler	ExponentialLR ($\gamma_{lr} = 0.95$)
Batch size	32
Epoche	120 (numero fisso, senza early stopping)

Tabella 5.2: Iperparametri utilizzati per l'addestramento della rete neurale.

La rete è stata addestrata sui due dataset generati, contenenti rispettivamente i grafi ER e BA, e producendo per ciascuno i seguenti modelli:

- 5 modelli nella versione Unrolling con numero di layer $T \in \{3, 5, 8, 12, 20\}$
- 1 modello nella versione Recurrent Unrolling con numero di layer $T = 20$

5.2.4 Protocollo di confronto

Il numero di layer T attraverso cui l'algoritmo viene srotolato dipende dalla dimensione del grafo. Per grafi con 50 nodi, gli autori [3] utilizzano $T = 20$ layer. Per confrontare le prestazioni in termini di GMSE dei 3 diversi approcci è stato deciso di strutturare la rete neurale con 20 layer e di fissare il numero massimo di iterazioni (max_iter) dell'Algoritmo 1 a 20.

Per l'Algoritmo 1 i parametri α, β, γ sono stati impostati tramite grid search su un numero di iterazioni dell'algoritmo pari a 20.

Allo stesso modo si è proceduto per le restanti configurazioni del numero di layer/i-terazioni T .

5.3 Risultati ricostruzione grafi ER

Modello	Iterazioni / layer	GMSE
PDS (tarato a 20 iter)	20	0.2356
PDS (tarato a convergenza)	500	0.1419
Recurrent Unrolling	20	0.1601
Unrolling	20	0.0745

Tabella 5.3: GMSE sul test set nella ricostruzione di grafi ER.

I risultati ottenuti per la ricostruzione di grafi ER dimostrano come l’approccio Unrolling riduca il GMSE del 47.5% rispetto a PDS su 500 iterazioni (a convergenza). A parità di numero di iterazioni/layer, Recurrent Unrolling raggiunge un GMSE più basso rispetto a PDS, tuttavia ottiene un risultato peggiore rispetto a PDS a convergenza.

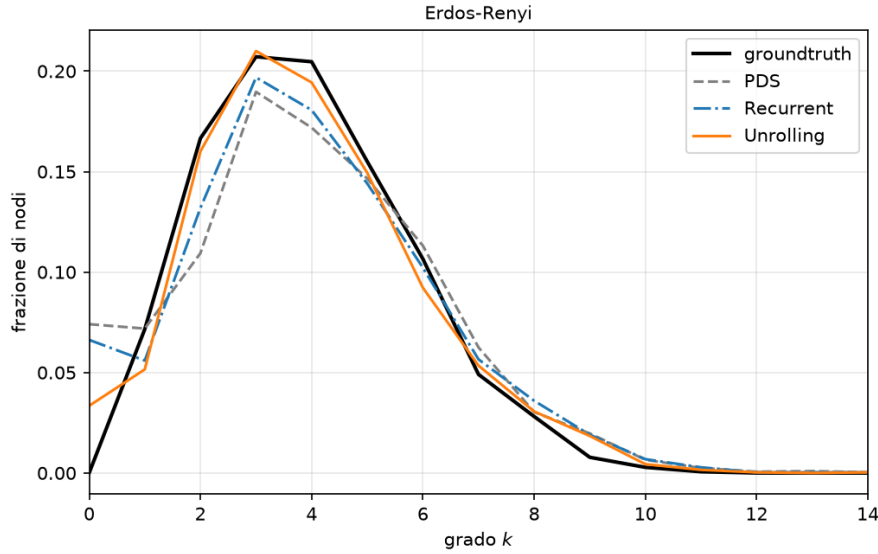


Figura 5.2: Confronto della distribuzione dei gradi su grafi ER

La Figura 5.2 mostra la distribuzione dei gradi aggregata sui 64 grafi di test, dopo che ciascuna stima è stata sparsificata mantenendo i K archi di peso maggiore. In questo modo tutte le ricostruzioni possiedono lo stesso numero di archi e il medesimo grado medio, e le differenze osservabili riguardano esclusivamente il modo in cui tali archi vengono distribuiti fra i nodi.

Il grafico evidenzia che le tre ricostruzioni seguono l’andamento del grafo di riferimento in modo sostanzialmente analogo. In assenza di eterogeneità nei gradi non vi è alcuna proprietà strutturale da recuperare, e i regolarizzatori analitici impiegati da PDS risultano ben allineati alla natura del problema.

5.4 Risultati ricostruzione grafi BA

Modello	Iterazioni / layer	GMSE
PDS (tarato a 20 iter)	20	0.2904
PDS (tarato a convergenza)	500	0.2573
Recurrent Unrolling	20	0.2558
Unrolling	20	0.0913

Tabella 5.4: GMSE sul test set nella ricostruzione di grafi BA.

I risultati mostrano come l'algoritmo Unrolling performi meglio di tutti gli altri approcci, apportando una riduzione del GMSE del 64.5% rispetto a PDS a convergenza. Invece, Recurrent Unrolling ottiene un GMSE più basso rispetto a PDS con sole 20 iterazioni, guadagno che si disperde con PDS a convergenza.

I dati raccolti mostrano come l'utilizzo di parametri indipendenti fra i layer provochi un salto prestazionale in termini di GMSE fra le due versioni di Algoritmo 2.

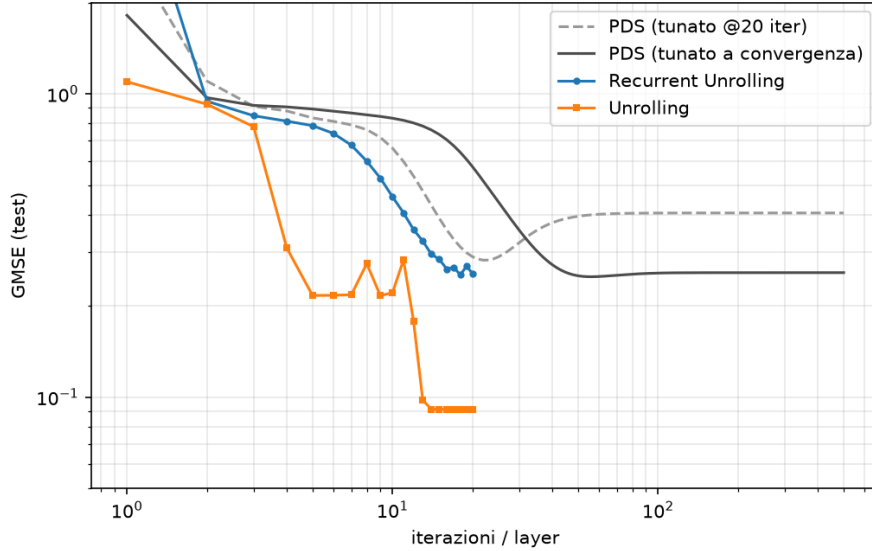


Figura 5.3: Andamento del GMSE sul test set in funzione del numero di iterazioni/layer.

La Figura 5.3 rappresenta il GMSE associato agli output intermedi generati dai modelli Unrolling e Recurrent Unrolling con $T = 20$, confrontandolo con i risultati ottenuti da PDS al crescere delle iterazioni. Come emerge chiaramente dalla Figura 5.3 a parità di iterazioni/layer, l'algoritmo Unrolling raggiunge l'ottimo molto più velocemente rispetto agli altri approcci. È interessante osservare come PDS non raggiunga mai l'Unrolling, nemmeno quando tarato a convergenza. Inoltre, il minimo non coincide con la convergenza, a dimostrazione dei limiti di espressività legati ai regolarizzatori fissi.

Altra osservazione interessante riguarda invece l'andamento del GMSE per PDS tarato su 20 iterazioni e a convergenza. Infatti, nel primo caso il GMSE raggiunge il suo minimo in prossimità della 22-esima iterazione, per poi risalire ed appiattirsi. Ciò dimostra come i parametri α, β, γ ottimi siano direttamente influenzati dal numero di iterazioni dell'algoritmo.

Un'ultima osservazione riguarda l'andamento dell'Unrolling. Come si può notare dal grafico, dopo la 14-esima iterazione la rete smette di migliorare e la motivazione è ancor più chiara dalla figura successiva.

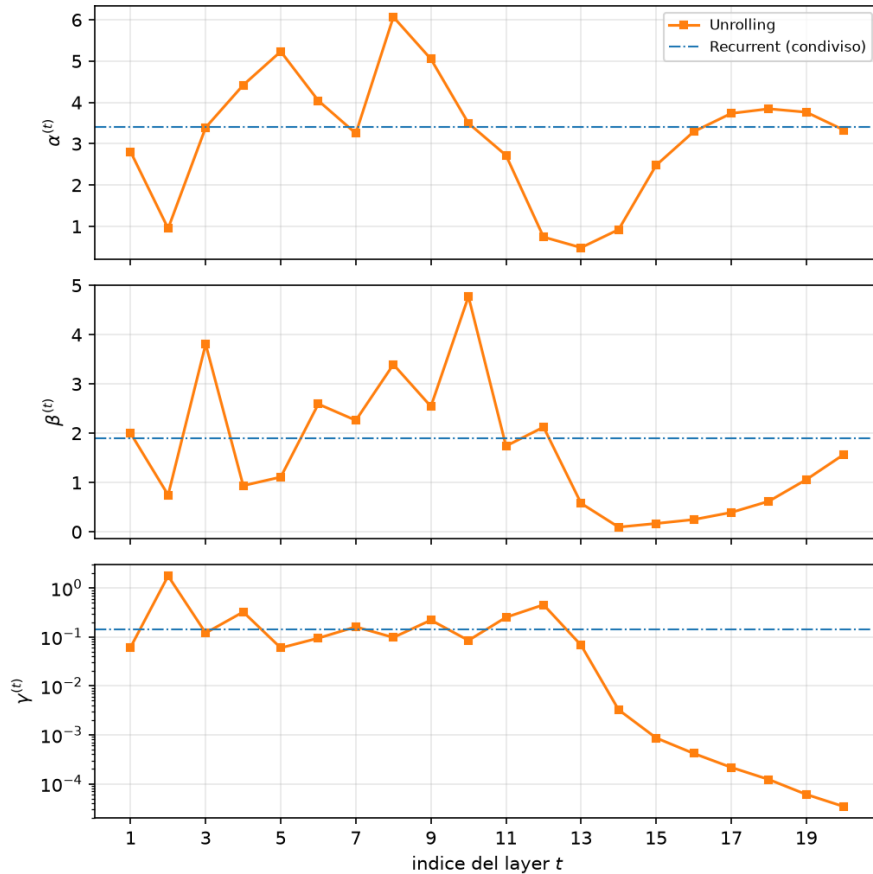


Figura 5.4: Valori appresi di $\alpha^{(t)}$, $\beta^{(t)}$ e $\gamma^{(t)}$ in funzione dell'indice del layer.

La Figura 5.4 mette a confronto i parametri α, β, γ costanti nel Recurrent Unrolling con i parametri $\alpha^{(t)}, \beta^{(t)}, \gamma^{(t)}$ appresi dalla rete per ciascuno dei 20 layer nell'Unrolling. Il guadagno di GMSE registrato dall'Unrolling rispetto al Recurrent Unrolling viene giustificato dal passaggio da 3 a 60 parametri addestrabili.

L'aspetto più interessante che emerge da questa figura riguarda lo step-size γ . Fino alla 13-esima iterazione il valore di γ oscilla intorno a 0.1, lo stesso valore individuato tramite grid search per PDS. Nelle iterazioni successive γ decresce monotonicamente rallentando fino ad interrompere l'apprendimento della rete. Questo fenomeno spiega il motivo per cui, nell'Unrolling, il GMSE non diminuisce più oltre la 14-esima iterazione, dimostrando di fatto che gli ultimi 6 layer della rete vengono spenti.

Il picco a $\gamma = 1.7$ al layer 2 è notevole: quasi 20 volte lo step size di PDS. Il modello fa un passo molto aggressivo all’inizio per uscire rapidamente da $w = 0$, cosa che un algoritmo con step fisso non può permettersi senza rischiare la divergenza. È un vantaggio strutturale dei parametri layer-wise.

Per valutare la ricostruzione dei grafi dal punto di vista topologico è stata calcolata la distribuzione dei gradi per ciascuno dei diversi approcci.

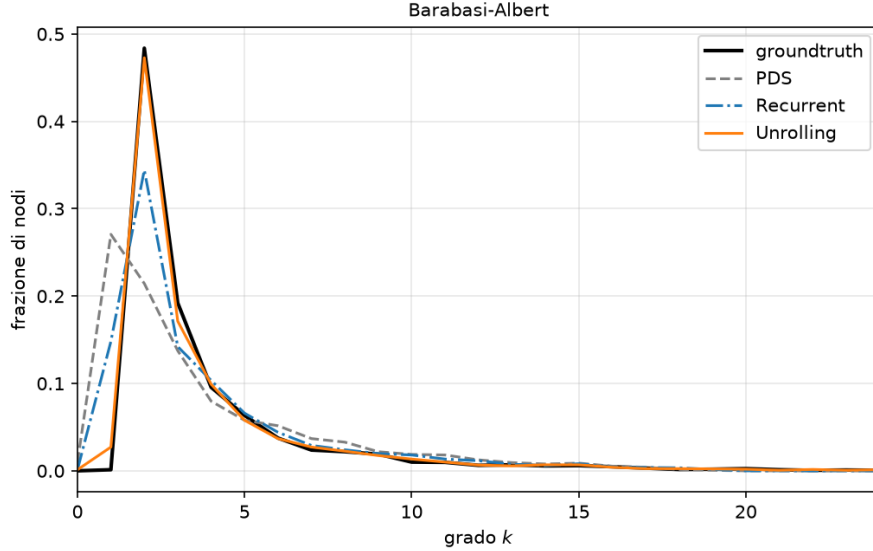


Figura 5.5: Confronto della distribuzione dei gradi su grafi BA

Il groundtruth presenta un picco pronunciato in $k = 2$, dove si concentra circa il 48% dei nodi, seguito da un decadimento rapido e da una coda sottile che si estende fino a $k \approx 24$. Il picco in $k = 2$ è una conseguenza diretta del modello generativo adottato: nel modello Barabási-Albert, con parametro di attaccamento pari a 2, ogni nuovo nodo entra nella rete con esattamente due archi. La coda estesa rappresenta invece i pochi hub che concentrano un numero elevato di connessioni.

L’Unrolling riproduce la distribuzione di riferimento in modo pressoché indistinguibile su tutto l’intervallo dei gradi. Il Recurrent Unrolling ne segue l’andamento generale ma sottostima il picco (0.34 contro 0.48). Invece, in PDS il picco è spostato in $k = 1$ e ridotto a 0.27, e la distribuzione risulta complessivamente più piatta. In particolare l’algoritmo produce un numero consistente di nodi di grado unitario, che nel grafo di riferimento sono di fatto assenti. Il fenomeno è coerente con la natura dei regolarizzatori impiegati: la barriera logaritmica $\alpha \mathbf{1}^T \log(\mathcal{D}\underline{w})$ impedisce la presenza di nodi isolati ma non impone alcun vincolo su un grado minimo. Il confronto evidenzia come, a fronte di un errore contenuto sui pesi, l’algoritmo model-based restituisca grafi la cui eterogeneità nei gradi è inferiore a quella reale.

I confronti precedenti fissavano il budget a $T = 20$ layer, quindi per capire se il vantaggio dell’Unrolling derivasse dalla profondità della rete sono stati addestrati

cinque modelli distinti, uno per ciascun valore $T \in \{3, 5, 8, 12, 20\}$. Simmetricamente, per ogni T è stata rieseguita la ricerca a griglia degli iperparametri di PDS con quello stesso numero di iterazioni, così che entrambi i metodi ricevano la migliore configurazione disponibile per il budget assegnato.

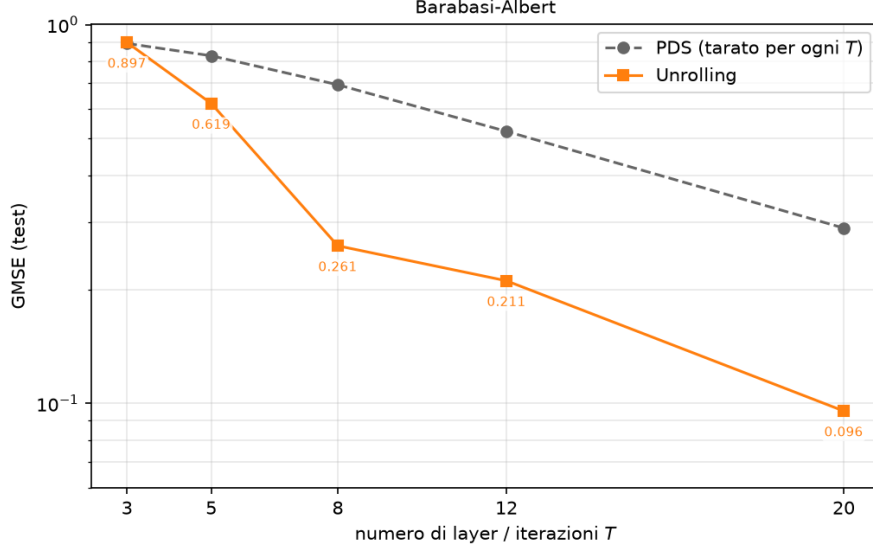


Figura 5.6: Confronto PDS e Unrolling su numero di layer/iterazioni crescenti

La Figura 5.6 mette a confronto il GMSE ottenuto sul test set dai cinque modelli Unrolling addestrati separatamente, uno per ciascun valore di T , con quello di PDS eseguito per lo stesso numero di iterazioni e con gli iperparametri ottimizzati per ciascun budget.

Le due curve partono sovrapposte a $T = 3$, con $GMSE \approx 0.90$ per entrambi, quindi in questo caso, la capacità di apprendimento non compensa l'insufficienza del budget. Da $T = 5$ le curve si allontanano progressivamente. PDS decresce in modo approssimativamente lineare, l'Unrolling scende molto più rapidamente, con un salto marcato fra $T = 5$ e $T = 8$ ($0.619 \rightarrow 0.261$) e un secondo fra $T = 12$ e $T = 20$ ($0.211 \rightarrow 0.096$). Dai risultati è chiaro che il divario cresce con la profondità: da un rapporto quasi unitario a $T = 3$ a un fattore superiore a 3 a $T = 20$. Il beneficio dell'apprendimento non è costante, ma scala in funzione del numero di parametri disponibili, che cresce linearmente con T ($3T$ contro i 3 fissi di PDS).

5.5 Confronto ricostruzioni grafi ER e BA

Modello	BA	ER
PDS (tarato a 20 iter)	0.2904	0.2356
PDS (tarato a convergenza)	0.2573	0.1419
Recurrent Unrolling	0.2558	0.1601
Unrolling	0.0913	0.0745
<i>riduzione del GMSE ottenuta dall'Unrolling</i>		
vs PDS (20 iter)	68.6%	68.4%
vs PDS (convergenza)	64.5%	47.5%
vs Recurrent Unrolling	64.3%	53.5%

Tabella 5.5: Confronto GMSE sul test set e riduzioni percentuali ottenute dall'Unrolling sulle due famiglie topologiche

La Tabella 5.5 confronta i risultati in termini di GMSE ottenuti dai diversi approcci applicati alle due diverse famiglie di grafi. Oltre a tali dati vengono riportate le riduzioni percentuali di GMSE apportate dall'Unrolling rispetto agli altri approcci.

La prima interessante osservazione riguarda la riduzione percentuale ottenuta dall'Unrolling rispetto a PDS su 20 iterazioni. A parità di budget (layer/iterazioni), il vantaggio è indipendente dalla topologia: 68.6% su BA e 68.4% su ER. La coincidenza è notevole e suggerisce che il guadagno in termini di GMSE non dipenda dalla presenza di una struttura nei grafi da apprendere, quanto piuttosto dalla maggiore flessibilità offerta dai coefficienti indipendenti per layer.

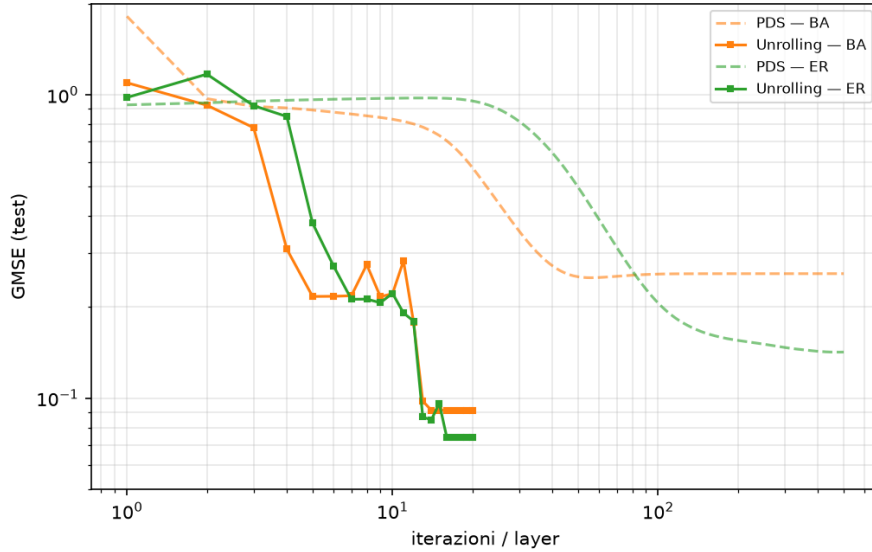


Figura 5.7: Confronto andamento del GMSE per grafi ER e BA

Dal confronto dei risultati emerge che PDS trae beneficio molto diverso dalle iterazioni aggiuntive. Passando da 20 a 500 iterazioni, PDS migliora del 40% su ER ($0.2356 \rightarrow 0.1419$) ma solo dell'11% su BA ($0.2904 \rightarrow 0.2573$). Su ER l'algoritmo continua a convergere verso una soluzione buona mentre su BA si arena, ciò è giustificato dai regolarizzatori impiegati. La norma L2 sui pesi e la barriera logaritmica sui gradi favoriscono soluzioni omogenee, che sui grafi ER coincidono sostanzialmente con la struttura vera, mentre sui BA no.

Per quanto riguarda il Recurrent Unrolling, i dati quantificano quanto la condivisione dei parametri risenta della struttura del grafo. Il divario Unrolling–Recurrent è 64.3% su BA e 53.5% su ER. Ciò significa che su ER i tre parametri condivisi bastano quasi, perché non c'è molto da differenziare fra layer; su BA la libertà fra i diversi layer conta di più.

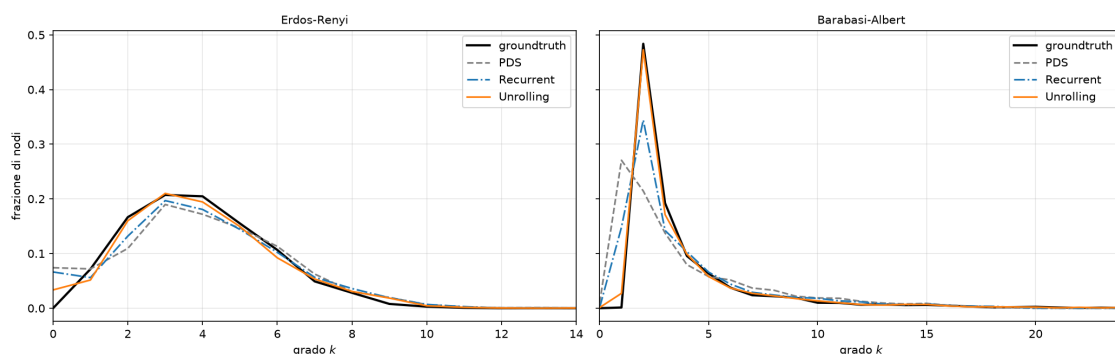


Figura 5.8: Confronto della distribuzione dei gradi per diverse famiglie di grafi

La Figura 5.8 affianca le distribuzioni dei gradi ottenute sulle due famiglie topologiche. Il confronto rende evidente come il comportamento dei tre algoritmi dipenda dalla presenza o meno di una struttura da recuperare.

Sui grafi Erdős–Rényi le quattro curve sono sostanzialmente sovrapposte, invece, sui grafi Barabási–Albert il quadro cambia. La distribuzione di riferimento presenta un picco pronunciato in $k = 2$, seguito da un decadimento rapido e da una coda estesa che rappresenta i nodi hub. L'Unrolling la riproduce in modo pressoché indistinguibile, il Recurrent Unrolling ne segue l'andamento ma sottostima il picco, mentre PDS se ne discosta in maniera sistematica, collocando il massimo in $k = 1$ e restituendo una distribuzione complessivamente più piatta, con un numero consistente di nodi di grado unitario, assenti nel groundtruth.

Il contrasto fra i due pannelli è il risultato più significativo del confronto. In assenza di eterogeneità nei gradi i regolarizzatori analitici impiegati da PDS risultano ben allineati alla natura del problema, e il vantaggio dei modelli srotolati si limita all'accuratezza sui pesi. Quando invece la topologia presenta una distribuzione a coda pesante, l'algoritmo model-based non è in grado di riprodurla e restituisce grafi la cui eterogeneità è inferiore a quella reale.

Si osservi come questa differenza non emerga dal solo GMSE, che assegna riduzioni pressoché identiche sulle due famiglie. La ricostruzione della struttura topologica

e quella dei pesi costituiscono dunque obiettivi distinti, e la scelta della metrica determina quale dei due venga effettivamente misurato.

Capitolo 6

Conclusioni

In questo lavoro sono stati implementati e confrontati tre approcci per l'apprendimento della topologia di un grafo a partire da osservazioni sui nodi: l'algoritmo di primal-dual splitting proposto in [1], il suo srotolamento in una rete neurale con parametri condivisi fra i layer (Recurrent Unrolling) e la variante con parametri indipendenti (Unrolling), entrambi ispirati al framework di [3]. La valutazione è stata condotta su grafi sintetici generati secondo i modelli Barabási–Albert ed Erdős–Rényi.

Il risultato principale conferma l'efficacia dell'approccio *Deep Unfolding*: a parità di budget computazionale, l'Unrolling riduce il GMSE del 68.6% sui grafi scale-free e del 68.4% su quelli casuali rispetto all'algoritmo iterativo con iperparametri ottimizzati via grid search. Il divario non si annulla neppure facendo convergere PDS, a dimostrazione del fatto che il limite non è dunque di natura computazionale, ma riguarda l'espressività dei regolarizzatori analitici impiegati.

Il confronto fra la variante ricorrente e quella con parametri indipendenti quantifica il contributo della parametrizzazione layer-wise: il passaggio da tre a sessanta parametri addestrabili produce una riduzione ulteriore del GMSE del 64.3% sui grafi BA. L'analisi dei coefficienti appresi mostra come tale libertà venga impiegata per un passo iniziale marcatamente più aggressivo di quanto un algoritmo a passo fisso possa permettersi.

In termini di GMSE, l'Unrolling ottiene un vantaggio pressoché identico sulle due famiglie. Invece, il confronto delle distribuzioni dei gradi mostra che sui grafi ER tutti gli algoritmi riproducono fedelmente la distribuzione di riferimento, mentre sui grafi BA soltanto i modelli srotolati ne conservano l'eterogeneità, laddove PDS restituisce strutture più omogenee. La ricostruzione dei pesi e quella della struttura topologica costituiscono dunque obiettivi distinti, e la sola metrica di errore quadratico non è sufficiente a caratterizzare la qualità di una stima.

Il presente lavoro si presta a diversi sviluppi futuri:

- implementazione del modulo TopoDiffVAE proposto in [3], che sostituisce l'operatore prossimale con un autoencoder variazionale, fornendo maggiore

espressività nella cattura delle proprietà topologiche dei grafi.

- analisi su grafi di dimensioni maggiori e con segnali che si discostano dal modello gaussiano assunto dalla formulazione.

Riferimenti

- [1] Vassilis Kalofolias. How to learn a graph from smooth signals. In *Proceedings of the 19th International Conference on Artificial Intelligence and Statistics (AISTATS)*, volume 51 of *Proceedings of Machine Learning Research*, pages 920–929, Cadiz, Spain, 2016. PMLR.
- [2] Nikos Komodakis and Jean-Christophe Pesquet. Playing with duality: An overview of recent primal–dual approaches for solving large-scale optimization problems. *IEEE Signal Processing Magazine*, 32(6):31–54, 2015.
- [3] Xingyue Pu, Tianyue Cao, Xiaoyun Zhang, Xiaowen Dong, and Siheng Chen. Learning to learn graph topologies, 2021.